Instruction Induction: From Few Examples to Natural Language Task Descriptions

Or Honovich[τ] **Uri Shaham**[τ] **Samuel R. Bowman**[ν] **Omer Levy**[τμ]

[τ] Tel Aviv University
[ν] New York University
[μ] Meta AI

Abstract

Large language models are able to perform a task by conditioning on a few input-output demonstrations – a paradigm known as *in-context learning*. We show that language models can explicitly infer an underlying task from a few demonstrations by prompting them to generate a natural language instruction that fits the examples. To explore this ability, we introduce the *instruction induction* challenge, compile a dataset consisting of 24 tasks, and define a novel evaluation metric based on *executing* the generated instruction. We discover that, to a large extent, the ability to generate instructions does indeed emerge when using a model that is both large enough and aligned to follow instructions; InstructGPT achieves 65.7% of human performance in our execution-based metric, while the original GPT-3 model reaches only 9.8% of human performance. This surprising result suggests that instruction induction might be a viable learning paradigm in and of itself, where instead of fitting a set of latent continuous parameters to the data, one searches for the best description in the natural language hypothesis space.[1]

1 Introduction

Large language models (LMs) can perform unseen tasks by conditioning on a few labeled examples, effectively inferring the underlying tasks through a process known as *in-context learning* (Brown et al., 2020). However, task inference is implicit, and the ability of models to *explicitly* reason about it remains unexplored. In this work, we show that LMs can explicitly describe an underlying task, in natural language, given a few labeled examples.

We introduce the *instruction induction* challenge, in which a model is provided with a few input-output demonstrations, and is requested to generate a natural language instruction describing the connection between the input-output pairs. In our experiments, inducing instructions is done in a zero-shot manner by simply prompting the models to explain a small set of given demonstrations, as shown in Figure 1; we do not perform fine-tuning or use any labeled instruction induction data.

We examine instruction induction on 24 tasks, ranging from morphosyntactic tasks to style transfer and sentiment analysis. Since our goal is to shed light on the phenomenon of instruction induction, we focus on tasks that have clear and simple instructions. As a basic evaluation protocol, we collect human annotations and use them as gold-standard references; the generated instructions are then compared to these references using BERTScore (Zhang et al., 2020). Moreover, we suggest a novel evaluation metric for instruction induction: *execution accuracy*. The execution accuracy of a generated instruction is measured by testing whether LMs can correctly perform the task in a zero-shot manner by using the generated instruction alone, without any demonstrations.

Our experiments reveal a surprising ability at generating correct instructions. The best-performing model, InstructGPT (Ouyang et al., 2022), achieves an average BERTScore of 44.4, compared to human performance of 60.0; when measuring execution accuracy, the model reaches 43.6, with human-written instructions reaching 66.4. For some tasks, the model's performance is on par or even better than human performance. When qualitatively examining the generated instructions, we often observe accurate instructions, even for some of the more challenging tasks. For instance, in the task of formality style transfer, generated instructions include "Translate the inputs into more formal language" and "Use formal language". For semantic text similarity, the generated instructions include "For each input, rate the similarity of the two sentences on a scale of 0 to 5, with 5 being a perfect match" and "Determine whether

[1] Our code and data are publicly available at https://github.com/orhonovich/instruction-induction

Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics
Volume 1: Long Papers, pages 1935–1952
July 9-14, 2023 ©2023 Association for Computational Linguistics

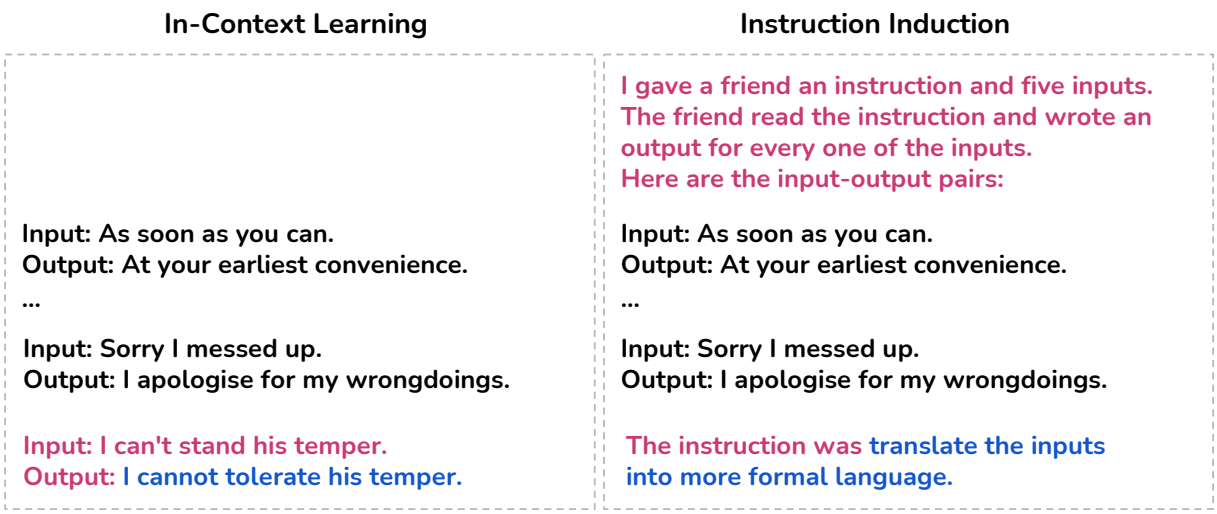


Figure 1: An example of instruction induction for the task of formality style transfer. *Left:* the standard in-context learning setting; given five demonstrations, complete the sixth. *Right:* instruction induction; the language model is prompted to generate a natural language instruction that describes the demonstrations. Model completions are in blue, prompt templates are in pink.

the two sentences are about the same thing".

Despite these impressive results, we find that this ability is currently unique to InstructGPT (Ouyang et al., 2022), which is both very large (175B parameters) and was especially fine-tuned to follow instructions. Ablations on smaller versions of InstructGPT as well as the original 175B-parameter GPT-3 (Brown et al., 2020) yield dramatically weaker performance. These findings are in line with recent work showing that increasing model size unlocks new capabilities (Chowdhery et al., 2022; Ganguli et al., 2022), and serves as additional evidence for the strength of instruction tuning (Sanh et al., 2022; Wei et al., 2022a; Ouyang et al., 2022), perhaps even pointing to the necessity of complementing standard next-word prediction with additional objectives.

The fact that models can induce natural language instructions suggests that instruction induction may serve as a learning paradigm of its own, where the optimization goal is to find the best natural language description that fits the observations. In this ambitious view of instruction induction, natural language can function as the hypothesis space, and a model is required to learn a natural language rule describing the relation between inputs and outputs in the training examples, rather than a set of uninterpretable parameters. While we currently provide a proof-of-concept for that idea, extending it by grounding models in natural language has the immediate benefit of human interpretability, explainability, and verifiability, while potentially alleviating overfitting and other issues associated with spurious correlations.

2 Instruction Induction

We begin by formulating the task of instruction induction. Given a sequence of n demonstrations $\{x_k, y_k\}_{k\in\{1,\ldots,n\}}$, the goal is to generate a *single* natural language instruction, such that for each x_k, following the instruction results in y_k. This format is similar to in-context learning (Brown et al., 2020), only here the desired output is an *instruction* describing the relation between the inputs and outputs of the demonstrations. We require models to perform this in a zero-shot setting, without any fine-tuning on labeled data. Figure 1 illustrates the difference between standard in-context prompting and instruction-induction prompting.

To elicit models to generate instructions, we consider prompts that would elicit humans to do so. We design a meta-prompt presenting instruction induction as a challenge puzzle and verify its clarity in a human study (§3.3). The prompt is presented in Figure 1 (right side, in pink).[2]

While prior work already shows that large LMs are often able to infer a latent task from a given set of demonstrations, this has been largely based on their ability to *execute* the task on a held-out exam-

[2]We found this prompt informative for both humans and models in preliminary experiments. We provide a meta-prompt analysis in Appendix C.

ple. Instruction induction requires that the model *describe* the underlying task in natural language.

3 Data

We evaluate on 24 tasks. Example tasks are listed in Table 1. See Table 4 in Appendix A for the full list of tasks. We select these tasks as they vary in difficulty and represent different aspects of language understanding, ranging from surface-level spelling to sentence similarity and causality detection.[3] Since our primary goal is to study the phenomenon of instruction induction under lab conditions, we focus on tasks that have simple instructions and defer tasks with more complicated instructions for future work. We review the dataset's format, the annotation and verification processes we conducted to ensure that the tasks are viable, and finally discuss a theoretical limitation of this setup.

3.1 Format

In every task, each single *demonstration* (x_k, y_k) is formatted as follows:

Input: x_k
Output: y_k

For instance, one demonstration in the pluralization task is "Input: cat" followed by "Output: cats" in a new line. We split each task's demonstrations into two sets: an *induce* set, which we use for generating instructions, and an *execute* set, which is held out for the execution accuracy evaluation metric (see §4.2). Each *instruction induction example* is composed of 5 demonstrations sampled randomly without replacement from the induce set, concatenated with new-line separators; we create 100 examples for each task. When generating instructions, each example is placed inside the instruction induction prompt, and fed to the model (Figure 1, right).

3.2 Annotating Reference Instructions

We collect 10 gold-reference human-annotated instructions via college-graduate English-speaking annotators. For each task, we provide the annotators with the exact same input we intend to provide a model: 5 input-output demonstrations wrapped by the instruction-induction prompt (Figure 1). We manually verify each annotation and discard ones that do not correctly describe the task. We refer to this set of annotations as the *gold* annotations, and use them for reference-based evaluation (see §4).

[3]See Appendix A for the full details of each task.

3.3 Verification

Prior to the instruction induction experiments, we conduct two tests to ensure that either models or humans can infer the underlying task given 5 demonstrations. We first verify that models can indeed execute our tasks given 5 demonstrations using in-context learning. Secondly, we conduct a human study to confirm that 5 demonstrations are enough for humans to describe the latent tasks.

In-Context Learning We prompt models with 5 input-output demonstrations and concatenate an additional test input x_{k+1}, and verify that the models are able to correctly predict y_{k+1} (Figure 1, left). For each task, we repeat this experiment 100 times, each with a different set of demonstrations and test inputs. We do not provide the model with any instruction beyond the "Input: x_k Output: y_k" format. We evaluate each task using its predefined evaluation metric.[4] The in-context results for GPT-3 (Brown et al., 2020) and InstructGPT (Ouyang et al., 2022) (see model details in §5) are reported in Table 5 in Appendix B, which shows that in-context learning can reach 80% accuracy and above on most tasks.

Human Study To assess the human ability to induce instructions, we collect human-written instructions, using annotators that *did not* participate in the gold references collection. As in the gold-reference annotation process, we provide annotators with the same input we intend to provide to models. We refer to this set of annotations as the *control* annotations. We then manually count, for each task, the number of annotators that provided a correct instruction, and report the correct instructions percentage in Table 5 (Appendix B). In all but one task (*Larger Animal*), at least 4 out of 5 annotators were able to produce correct task descriptions.

We also use the control group's annotations to establish a human baseline for automatic evaluation metrics. For reference-based evaluation (§4.1), we treat the control annotations as generated instructions and compare them against the gold annotations, while for execution accuracy (§4.2), we use the control annotations to measure human performance, and the gold references as a ceiling metric.

[4]All metrics are variants of simple string matching, with some task-specific heuristics, for example, to allow for multiple correct answers. See Appendix A for exact details.

Category	Task	Instruction	Demonstration
Spelling	First Letter	Extract the first letter of the input word.	cat → c
Syntax	Negation	Negate the input sentence.	Time is finite → Time is not finite.
Lexical Semantics	Antonyms	Write a word that means the opposite of the input word.	won → lost
Phonetics	Rhymes	Write a word that rhymes with the input word.	sing → ring
Semantics	Cause Selection	Find which of the two given cause and effect sentences is the cause.	Sentence 1: The soda went flat. Sentence 2: The bottle was left open. → The bottle was left open.
	Common Concept	Find a common characteristic for the given objects.	guitars, pendulums, neutrinos → involve oscillations.
Style	Formality	Rephrase the sentence in formal language.	Please call once you get there → Please call upon your arrival.
Numerical	Sum	Sum the two given numbers.	22 10 → 32
Multi-lingual	Translation	Translate the word into German / Spanish / French.	game → juego
GLUE	Sentiment Analysis	Determine whether a movie review is positive or negative.	The film is small in scope, yet perfectly formed. → positive
	Sentence Similarity	Rate the semantic similarity of two input sentences on a scale of 0 - definitely not to 5 - perfectly.	Sentence 1: A man is smoking. Sentence 2: A man is skating. → 0 - definitely not

Table 1: Example tasks used in our instruction-induction experiments. For each task, we show a corresponding instruction and demonstration, with → separating the input from the output.

3.4 Ambiguity

A theoretical challenge in inducing instructions is ambiguity. For example, when given the single demonstration "Input: The coffee is too hot. Output: The, too, hot", one could infer that the underlying task is either "write all the words containing the letter T" or "write all the three-lettered words", both valid interpretations. Ambiguity might confuse models tasked with instruction induction while also making evaluation less reliable. In practice, providing 5 demonstrations typically resolves the ambiguity in our set of tasks. As evident from the data verification process, our tasks can typically be inferred by models and/or humans.

Inducing more complex task descriptions, such as predicting detailed annotation guidelines, may pose a greater challenge in terms of ambiguity. We hypothesize that providing more than 5 demonstrations could mitigate some of that challenge, and leave further exploration of this avenue to future work.

4 Evaluating Generated Instructions

As a standard text generation metric, we report BERTScore (Zhang et al., 2020). However, the instruction induction challenge has a unique property, which does not usually hold for other text generation tasks: the instructions are *executable*. Their correctness can therefore be measured directly by utilizing them as prompts.

4.1 Reference-Based Evaluation

We use BERTScore (Zhang et al., 2020) to compare the model-generated instructions against the collected gold annotations. As mentioned in §3.2, we use only the correct, verified annotations as references. We take the maximal BERTScore-F1 over all gold-reference annotations to account for natural variations in instruction formulation.[5] We also establish a human baseline for each task using the *control* annotations, which were collected from a separate control group of annotators (§3.3), which we compare against the *gold* annotations in exactly the same way as model-generated instructions. In preliminary studies, we experiment with other reference-based metrics (ROUGE and BLEU), and find BERTScore to be a better predictor of instruction quality, although all metrics showed similar trends.

[5]We use BERTScore version 0.3.11 with the DeBERTa-xl-MNLI model (He et al., 2021; Nangia et al., 2017).

4.2 Execution Accuracy

We introduce *execution accuracy*, a new metric unique to the instruction induction task. We define a correct instruction as one that can guide humans to produce the expected output. To approximate human behavior, we use an instruction-tuned model and test whether it can follow the generated instruction. Concretely, to measure the execution accuracy of a predicted instruction I (e.g., "Write the plural form of the given word.") for a task T (pluralization), we prompt a model with I and an input x ("cat"). We then test, given I and x, whether the model can correctly predict y, the output of performing T on the input x (*cats*).

To obtain meaningful results, we measure execution accuracy on the 100 held-out *execute* examples for each task. The execution accuracy of an instruction I is therefore computed by taking the average over $Score_T(I(x_n), y_n)$ for all x_n in the *execute* set, where $Score_T$ denotes the task's corresponding metric (see Appendix A), and $I(x_n)$ is the result of prompting a predefined language model with the instruction I and the input x_n. As recent models are trained to follow instructions (Sanh et al., 2022; Wei et al., 2022a; Ouyang et al., 2022), and due to the relative clarity of our tasks, we expect correct instructions to yield high execution accuracy when using a sufficiently powerful execution model.[6]

5 Results

Baseline Models We experiment with eight versions of GPT-3 (Brown et al., 2020), a Transformer decoder language model. First, we experiment with the most current version available in the OpenAI API, for each of the four available model sizes. Though not stated explicitly in the API, we assume these models are those reported by Ouyang et al. (2022), and we therefore refer to them as *Instruct* models.[7] We also experiment with the four originally published GPT-3 versions.[8] By default, we refer to the largest Instruct model as *InstructGPT*, and the original 175B-parameter model as *GPT-3*. All model generations were produced using the greedy decoding algorithm.

[6]Execution accuracy has been used to evaluate code generation (Yu et al., 2018). Here, we use execution accuracy to evaluate *natural language* instructions, with a strong language model playing the role of the interpreter.

[7]Concretely, we use: text-davinci-002, text-curie-001, text-babbage-001, text-ada-001.

[8]davinci, curie, babbage, ada.

Model	BERTScore	Execution
GPT-3		
Ada	-7.7	4.0
Babbage	4.1	3.2
Curie	13.9	7.9
DaVinci	14.6	6.5
InstructGPT		
Ada	5.9	4.4
Babbage	-0.5	3.8
Curie	10.7	8.8
DaVinci	44.4	43.6
Human (Control)	60.0	66.4

Table 2: Average BERTScore and execution accuracy across tasks. BERTScore is measured against the gold references. The execution accuracy for all generated instructions is measured using InstructGPT as the execution model. Human performance is measured using the human control group's instructions.

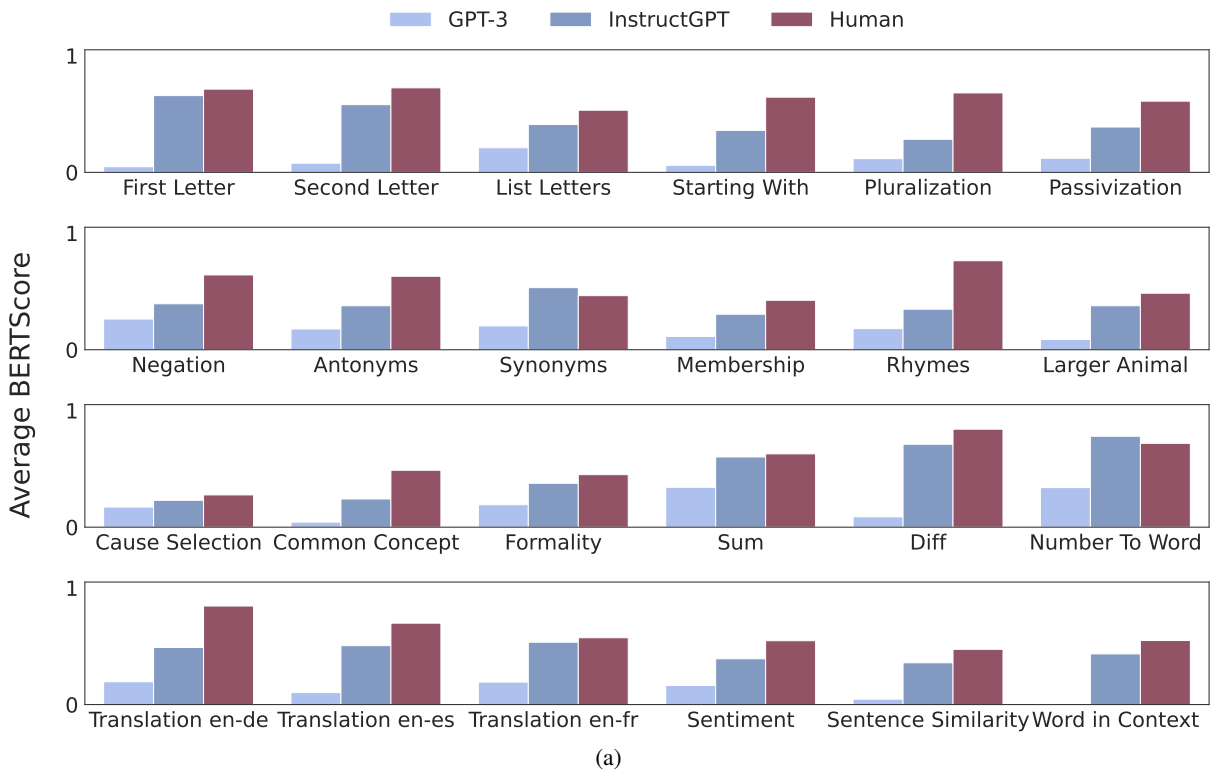
5.1 Comparing to Gold Annotations

Figure 2a presents the average BERTScore per task (see §4.1). Results show that the InstructGPT model has, to some extent, the ability to induce instructions from a few demonstrations; in 13 out of 24 tasks it achieves at least 75% of human performance. GPT-3, on the other hand, is quite far from human performance across the board.

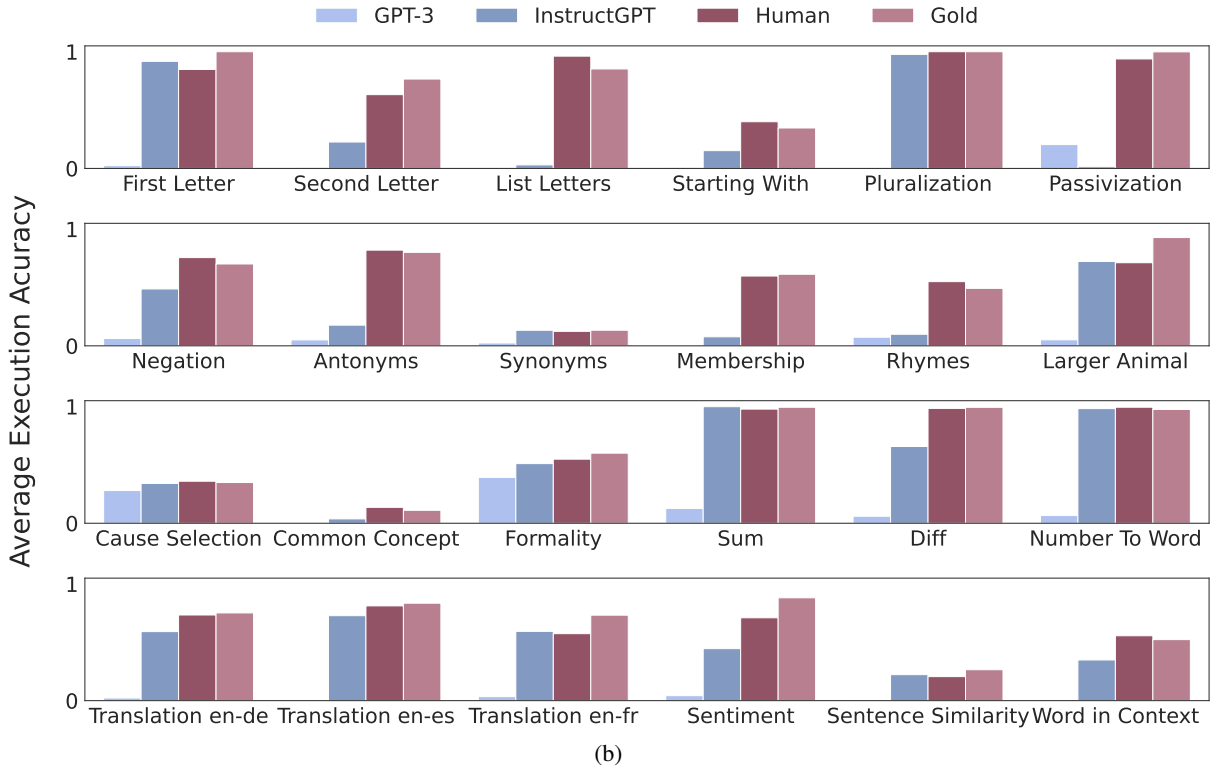
Table 2 shows the average scores across all tasks. We observe the same trend; while InstructGPT's BERTScore is 15.6 points lower than human performance, the gap between GPT-3 and humans is 45.4 points. Moreover, we observe that smaller models – even those fine-tuned to follow instructions – do not exhibit any instruction-induction abilities. Scores are slightly higher for larger models of the same family (except for the InstructGPT-Babbage outlier), but are overall low. Excluding the largest models, there does not appear to be a significant advantage for Instruct models over the originals when controlling for model size.

5.2 Execution Accuracy

We compute the execution accuracy as detailed in §4.2, and report the average over 100 generated instructions for each task. As an execution model, we use the largest InstructGPT model. We also use this model to induce instructions, and while using it as an execution model might bias results towards its own generations, preliminary experiments show that no other model is as good at following instructions as InstructGPT. As a point of reference, we



(a)



(b)

Figure 2: (a) Average BERTScores of model-generated instructions for each task, compared to the performance of the control group's manually-authored instructions. The BERTScore for each instruction is computed using the human *gold* annotations as references. (b) Average execution accuracy of model-generated instructions for each task, compared to the execution accuracy measured for human-written instructions. The *Human* baseline is measured by taking the control group's annotations, while the *Gold* ceiling metric is based on the separately-annotated and verified gold annotations.

apply the execution accuracy evaluation protocol to human-written instructions. First, to compare models with human performance, we measure the execution accuracy of the *control* annotation set. Second, to account for limitations in the execution model, we measure execution accuracy of the correct (manually verified) *gold* annotations, which acts as an approximated ceiling metric.

Figure 2b presents the execution accuracy per task. In 12 out of 24 tasks, InstructGPT achieves at least 75% of the execution accuracy measured for the human-written instructions. GPT-3 shows much weaker execution accuracy, scoring less than 10% on 20 of the 24 tasks. In fact, only in the cases of formality, passivization, and cause selection does it approach human performance, and that is largely an artifact of a more lenient evaluation metric in the case of formality and cause selection, or due to the execution model being right for the wrong reasons in the case of passivization (see §6). In some tasks, the control annotations are of high quality and reach a higher score than the verified gold annotations, likely due to variance of the execution model in such cases.

Table 2 shows the same trends. On average, InstructGPT achieves 65.7% of human performance, while GPT-3 reaches only 9.8% of human performance. When considering different model families or sizes, we do not see any substantial improvements when increasing model size or adding instruction tuning, with the exception of the largest InstructGPT model. The ability to generate instructions seems to only emerge when a model is both large enough and aligned to follow instructions. Overall, even the best-performing model still does not reach human performance, leaving room for future improvement.

6 Analysis

To gain further insight into the successes and failures of instruction induction prompting, we manually analyze the model-generated instructions of 5 tasks. Table 3 shows the most common predictions of GPT-3 and InstructGPT for each of these tasks.

InstructGPT obtains high, or close to human execution accuracy scores for three of these tasks (*First Letter*, *Sentence Similarity*, *Pluralization*). Indeed, the instructions for both *First Letter* and *Sentence Similarity* accurately describe the task. However, the instruction generated for *Pluralization* is not entirely precise, since it dismisses other forms of pluralization such as -es, -ies, and irregulars. Although the instruction only asks to add an "s", the execution model often ignores the specifics and produces the correct plural form; in one case, the input word was "life" and the output was "lives". While this particular instruction accounts for 24% of the induced instructions in the pluralization task, some predictions do explicitly mention pluralization, though not always accurately, e.g., "Add -s to the end of each word to make it plural".

For some tasks, InstructGPT fails to produce accurate instructions, even if it is able to solve via in-context learning (see Table 5). In *Passivization*, 98% of the predicted instructions were to simply "reverse the order of the subject and object", while ignoring additional surface-form manipulations needed to convert the given sentence into passive form; e.g., for the input "The authors supported the scientist", following the instructions produces the output "The scientist supported the authors", while the correct passive form is "The scientist was supported by the authors". Surprisingly, the instructions generated by GPT-3 obtained higher execution accuracy than the InstructGPT, even though they were entirely unrelated. In 24% of the cases, GPT-3 predicted "The friend wrote the following output:" – an instruction that apparently prompts the execution model to often rephrase the input in passive form. Lastly, in *Antonyms*, 60% of InstructGPT's predictions were "Reverse the input", and another 11% were "Reverse the word". While one could imagine an interpretation of these instructions that reflects the task (reversing the *meaning* of the word), the execution model interprets them literally, and reverses the input words' letters.

Overall, GPT-3 did not exhibit any instruction induction abilities, although it did often phrase outputs in imperative language. One relatively common prediction was the generic instruction "Write an output for every input". Because these empty instructions are in the right format, they tend to have some overlap with the reference instructions, which inflates their BERTScore. Execution accuracy, on the other hand, is robust to this phenomenon, and typically assigns GPT-3's outputs very low scores.

7 Related Work

In-Context Learning Brown et al. (2020) suggest that models can learn a task by conditioning on few input-output demonstration pairs, without any fine-tuning or gradient updates. This paradigm,

Task	GPT-3	InstructGPT
First letter	The friend's output was:	Write the first letter of each word.
Sentence Similarity	The friend wrote the following output:	For each input, rate the similarity of the two sentences on a scale of 0 to 5, with 5 being a perfect match.
Pluralization	The friend's output was:	Add 's' to the end of each word.
Passivization	The friend wrote the following output:	Reverse the order of the subject and the object in the sentence.
Antonyms	The friend's output was:	Reverse the input.

Table 3: Examples of the instructions generated by GPT-3 and InstructGPT for five of our tasks.

known as *in-context learning* or *prompt-based learning* (Liu et al., 2021), has been the focus of many research efforts lately: Du et al. (2021) suggest methods for more efficient in-context learning, Zhao et al. (2021) study methods for improving the stability and accuracy of prompt-based models, Chen et al. (2021) and Min et al. (2022a) conduct meta-training with an in-context learning objective, while other work studies the effect of the provided prompts (Reynolds and McDonell, 2021; Webson and Pavlick, 2021; Min et al., 2022b), or suggests prompt reframing techniques (Mishra et al., 2021) and prompt retrieval methods (Rubin et al., 2021). To the best of our knowledge, all previous work study in-context learning through the lens of *executing* a latent task, while we focus on the ability to explicitly *describe* it.

The Instruction Paradigm Efrat and Levy (2020) propose to learn new tasks from natural language instructions. Mishra et al. (2022) and Wang et al. (2022b) collect crowdsourcing instructions used to create NLP datasets into a benchmark for measuring the ability to solve tasks by reading instructions. Recent work shows that fine-tuning on task instructions (*instruction tuning*) improves the zero-shot learning abilities of LMs (Sanh et al., 2022; Wei et al., 2022a; Ouyang et al., 2022). Prasad et al. (2022) introduce an edit-based search approach for improving existing instructions used for prompting. In this work, we focus on models' ability to *generate* instructions, rather than their ability to *execute* instructions written by humans.

Intermediate Reasoning Steps Nye et al. (2022) show that LMs can perform complex computations by writing intermediate steps on a "scratchpad". In *chain of thought prompting* (Wei et al., 2022b), input-output demonstrations are enriched with sentences elaborating intermediate task reasoning steps, improving the performance of LMs on tasks requiring reasoning skills. Subsequent work further improves the performance on such tasks using a *self-consistency* ensemble (Wang et al., 2022a), which samples a set of diverse chain-of-thought reasoning paths, taking the majority vote over all generated answers. Zelikman et al. (2022) utilize a small set of examples labeled with chain-of-thought rationales and a large set of unlabeled data to iteratively bootstrap automatic rationale generation, thus creating a large dataset labeled with such rationales to enable fine-tuning. In contrast, we study the ability of LMs to generate a description of the task, rather than generating intermediate reasoning steps as a means of executing complex tasks.

Learning a Natural Language Hypothesis Zhong et al. (2022) propose to automatically describe the differences between two data distributions D_0 and D_1 by finding a description that is more true for D_1, e.g., "is military related" or "is longer in sentence length". They frame this task as learning a natural language hypothesis. In this work, we suggest describing a task based on demonstrations of this task alone, rather than describing the differences between two data distributions.

8 Discussion

This work demonstrates that large LMs can not only infer new tasks based on a handful of demonstrations, but also describe them in natural language. We provide evidence of this ability on a diverse set of language tasks, and show that while instruction induction abilities are limited to a single state-of-the-art model, this model does indeed approach human performance on about half the tasks.

It is not unreasonable to assume that models in the near future will be even better at processing human-generated instructions, and it is therefore interesting to discuss the potential applications of

instruction induction. In particular, we envision a use case in which instruction induction serves as a machine learning approach; instead of converting a dataset into a set of continuous parameters, we could produce a natural language instruction that best describes the data. Grounding the model in concise natural language has the advantage of interpretability, and has the potential to solve fundamental issues pertaining to spurious correlations. While it is still too early to determine whether this approach is viable, we view it as an intriguing direction for future research.

9 Limitations

Since our primary goal is to study the phenomenon of instruction induction under lab conditions, we focus on tasks that have simple instructions. Future work may extend instruction induction research by including tasks with more complex instructions. These tasks are expected to pose a greater evaluation challenge, especially when considering reference-based methods. Evaluating through execution accuracy, however, may mitigate some of that challenge. Additionally, only one model showed instruction induction abilities, i.e., text-davinci-002. The exact implementation details of the model and its training data are not publicly available, thus we are unable to investigate the reason behind the emergence of this ability. However, we note that our goal is to present the phenomenon of instruction induction and to raise the ambitious possibility of instruction induction as a learning paradigm. Thus, our goal is not to focus on specific models but rather to shed light on this unexplored phenomenon. Finally, we point to a limitation of the execution accuracy metric, namely assuming the existence of a good-enough instruction-tuned model. Due to recent interest and progress in instruction tuning, we believe this to be a reasonable assumption.

Ethics Statement

We believe that inducing instructions, as well as grounding in natural language in general, can potentially improve interpretability and explainability. We therefore view this line of research as having a positive effect on the ability to avoid unwanted artifacts.

References

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.

Daniel Cer, Mona Diab, Eneko Agirre, Iñigo Lopez-Gazpio, and Lucia Specia. 2017. SemEval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 1–14, Vancouver, Canada. Association for Computational Linguistics.

Yanda Chen, Ruiqi Zhong, Sheng Zha, George Karypis, and He He. 2021. Meta-learning via language model in-context tuning.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. Palm: Scaling language modeling with pathways.

Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten Bosma, Zongwei Zhou, Tao Wang, Yu Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathy Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. 2021. Glam: Efficient scaling of language models with mixture-of-experts.

Avia Efrat and Omer Levy. 2020. The turking test: Can language models understand instructions?

Hady Elsahar, Pavlos Vougiouklis, Arslen Remaci, Christophe Gravier, Jonathon Hare, Frederique Laforest, and Elena Simperl. 2018. T-REx: A large scale alignment of natural language with knowledge base triples. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*, Miyazaki, Japan. European Language Resources Association (ELRA).

C. Fellbaum. 1998. *WordNet: An Electronic Lexical Database*. MIT Press.

Deep Ganguli, Danny Hernandez, Liane Lovitt, Nova DasSarma, Tom Henighan, Andy Jones, Nicholas Joseph, Jackson Kernion, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Nelson Elhage, Sheer El Showk, Stanislav Fort, Zac Hatfield-Dodds, Scott Johnston, Shauna Kravec, Neel Nanda, Kamal Ndousse, Catherine Olsson, Daniela Amodei, Dario Amodei, Tom Brown, Jared Kaplan, Sam McCandlish, Chris Olah, and Jack Clark. 2022. Predictability and surprise in large generative models.

Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. 2021. Deberta: Decoding-enhanced bert with disentangled attention. In *International Conference on Learning Representations*.

Nora Kassner and Hinrich Schütze. 2020. Negated and misprimed probes for pretrained language models: Birds can talk, but cannot fly. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7811–7818, Online. Association for Computational Linguistics.

Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2021. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing.

Tom McCoy, Ellie Pavlick, and Tal Linzen. 2019. Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3428–3448, Florence, Italy. Association for Computational Linguistics.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations*.

Sewon Min, Mike Lewis, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2022a. MetaICL: Learning to learn in context. In *NAACL-HLT*.

Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2022b. Rethinking the role of demonstrations: What makes in-context learning work? *arXiv preprint*.

Swaroop Mishra, Daniel Khashabi, Chitta Baral, Yejin Choi, and Hannaneh Hajishirzi. 2021. Reframing instructional prompts to gptk's language.

Swaroop Mishra, Daniel Khashabi, Chitta Baral, and Hannaneh Hajishirzi. 2022. Cross-task generalization via natural language crowdsourcing instructions. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3470–3487, Dublin, Ireland. Association for Computational Linguistics.

Nikita Nangia, Adina Williams, Angeliki Lazaridou, and Samuel Bowman. 2017. The RepEval 2017 shared task: Multi-genre natural language inference with sentence representations. In *Proceedings of the 2nd Workshop on Evaluating Vector Space Representations for NLP*, pages 1–10, Copenhagen, Denmark. Association for Computational Linguistics.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. 2022. Show your work: Scratchpads for intermediate computation with language models. In *Deep Learning for Code Workshop*.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback.

Fabio Petroni, Tim Rocktäschel, Sebastian Riedel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, and Alexander Miller. 2019. Language models as knowledge bases? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China. Association for Computational Linguistics.

Mohammad Taher Pilehvar and Jose Camacho-Collados. 2019. WiC: the word-in-context dataset for evaluating context-sensitive meaning representations. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 1267–1273, Minneapolis, Minnesota. Association for Computational Linguistics.

Archiki Prasad, Peter Hase, Xiang Zhou, and Mohit Bansal. 2022. Grips: Gradient-free, edit-based instruction search for prompting large language models.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas. Association for Computational Linguistics.

Laria Reynolds and Kyle McDonell. 2021. Prompt programming for large language models: Beyond the few-shot paradigm. In *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*, CHI EA '21, New York, NY, USA. Association for Computing Machinery.

Ohad Rubin, Jonathan Herzig, and Jonathan Berant. 2021. Learning to retrieve prompts for in-context learning.

Victor Sanh, Albert Webson, Colin Raffel, Stephen Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Arun Raja, Manan Dey, M Saiful Bari, Canwen Xu, Urmish Thakker, Shanya Sharma Sharma, Eliza Szczechla, Taewoon Kim, Gunjan Chhablani, Nihal Nayak, Debajyoti Datta, Jonathan Chang, Mike Tian-Jian Jiang, Han Wang, Matteo Manica, Sheng Shen, Zheng Xin Yong, Harshit Pandey, Rachel Bawden, Thomas Wang, Trishala Neeraj, Jos Rozen, Abheesht Sharma, Andrea Santilli, Thibault Fevry, Jason Alan Fries, Ryan Teehan, Teven Le Scao, Stella Biderman, Leo Gao, Thomas Wolf, and Alexander M Rush. 2022. Multitask prompted training enables zero-shot task generalization. In *International Conference on Learning Representations*.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642, Seattle, Washington, USA. Association for Computational Linguistics.

Robyn Speer and Catherine Havasi. 2012. Representing general relational knowledge in ConceptNet 5. In *Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC'12)*, pages 3679–3686, Istanbul, Turkey. European Language Resources Association (ELRA).

Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. 2022. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615*.

Alon Talmor, Yanai Elazar, Yoav Goldberg, and Jonathan Berant. 2020. oLMpics-on what language model pre-training captures. *Transactions of the Association for Computational Linguistics*, 8:743–758.

Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2019. Superglue: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355, Brussels, Belgium. Association for Computational Linguistics.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022a. Self-consistency improves chain of thought reasoning in language models.

Yizhong Wang, Swaroop Mishra, Pegah Alipoormolabashi, Yeganeh Kordi, et al. 2022b. Benchmarking generalization via in-context instructions on 1,600+ language tasks. *arXiv*.

Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. 2018. Neural network acceptability judgments. *arXiv preprint arXiv:1805.12471*.

Albert Webson and Ellie Pavlick. 2021. Do prompt-based models really understand the meaning of their prompts?

Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V Le. 2022a. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. 2022b. Chain of thought prompting elicits reasoning in large language models.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921, Brussels, Belgium. Association for Computational Linguistics.

Eric Zelikman, Yuhuai Wu, and Noah D. Goodman. 2022. Star: Bootstrapping reasoning with reasoning.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. Bertscore: Evaluating text generation with bert. In *ICLR 2020*.

Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. 2021. Calibrate before use: Improving few-shot performance of language models. In *ICML*, pages 12697–12706.

Ruiqi Zhong, Charlie Snell, Dan Klein, and Jacob Steinhardt. 2022. Describing differences between text distributions with natural language. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 27099–27116. PMLR.

A Dataset Details

This appendix presents the full list of tasks (§A.1) and details each task's dataset (§A.2). Some datasets rely on a set of common English nouns (CEN), described at §A.3.

A.1 Full Dataset

Table 4 presents the full list of tasks used in our experiments.

A.2 Tasks

We elaborate on each task's data source, preprocessing protocol, and evaluation metric used in the in-context learning and execution accuracy experiments. As mentioned in §3, each task has *induce* and *execute* sets; unless stated otherwise, we sample 100 examples as the execute set for each task. When evaluating outputs, the generated text is first normalized; we take only the first generated sentence and lowercase it. We apply exact string match as the evaluation metric where applicable, elaborating only where alternative metrics are used.

First Letter In each demonstration, x_k is a noun, and y_k is the first letter of that noun. We construct the demonstrations by extracting the first letter of each word in CEN.

Second Letter Identical to the *First Letter* task, only here y_k is the second letter of x_k.

List Letters x_k is a noun from CEN, and y_k is a list of x_k's letters, separated by spaces.

Starting With x_k contains a sentence and a letter in brackets, and y_k lists the words in x_k that start with the given letter. We avoid cases in which y_k is empty, i.e., there is always at least one word in the input sentence starting with the given letter. Sentences are taken from the CoLA dataset (Warstadt et al., 2018). For the induce set, we create all (sentence, letter) pairs using CoLA's train set, and then sample 3,000 pairs. For the *execute* set, we create all (sentence, letter) pairs from CoLA's in-domain and out-of-domain dev sets, and then sample 50 in-domain and 50 out-of-domain examples. We evaluate using exact *set* match, by treating the output (and y_k) as a set of strings.

Pluralization Given a singular noun x_k, produce the plural form y_k. We take noun inputs from the CEN set, filtering out mass nouns using a predefined list.[9] To create the plural forms, we apply an automatic pluralization engine[10] and exclude nouns for which the engine's output did not appear at least 50 times in the Wikitext-103 corpus. This results in 2,043 singular-plural noun pairs.

Passivization Given a simple active sentence x_k, rephrase the sentence in passive voice y_k. We use the 1,000 HANS (McCoy et al., 2019) evaluation set active-passive entailed sentence pairs.

Negation y_k is the negation of the input sentence x_k. We use the negated LAMA dataset (Petroni et al., 2019; Kassner and Schütze, 2020), taking the 304 negated SQuAD (Rajpurkar et al., 2016) sentences, 300 ConceptNet (Speer and Havasi, 2012) sentences, 200 T-REx (Elsahar et al., 2018) sentences and 200 Google-RE[11] sentences. For ConceptNet and T-REx, we manually select these sentences to ensure their quality. For Google-RE, we automatically sample 100 sentences from the *place of birth* relation, and 100 from the *place of death* relation.

Antonyms y_k is the antonym of the input word x_k. We use the antonym pairs from oLMpics (Talmor et al., 2020), which were extracted from ConceptNet (Speer and Havasi, 2012) and WordNet (Fellbaum, 1998). For uniformity, we verify that all pairs are indeed antonyms according to WordNet.

Synonyms x_k is a word and y_k is its synonym. As in the antonyms task, we use the synonym pairs of Talmor et al. (2020). Since there can be multiple synonyms for each input word, the task's in-context and execution accuracy are evaluated by testing whether the gold answer (a single word) is contained in the predicted answer (which may be a list of words).

Membership x_k is a list of words, where some of the words represent animals, and y_k lists the animals from x_k. To construct the task's data, we first select 6 word categories: animals, clothing, colors, food, vehicles, and professions. We then take 10-50 words from each category, using only words that are categorized at the A1 or A2 levels according to the Common European Framework of

[9] https://gist.github.com/sudodoki/b5408fa4ba752cc22597250fc58a5970

[10] https://pypi.org/project/inflect/

[11] https://code.google.com/archive/p/relation-extraction-corpus/

Category	Task	Instruction	Demonstration
Spelling	First Letter	Extract the first letter of the input word.	cat → c
	Second Letter	Extract the second letter of the input word.	cat → a
	List Letters	Break the input word into letters, separated by spaces.	cat → c a t
	Starting With	Extract the words starting with a given letter from the input sentence.	The man whose car I hit last week sued me. [m] → man, me
Morpho-syntax	Pluralization	Convert the input word to its plural form.	cat → cats
	Passivization	Write the input sentence in passive form.	The artist introduced the scientist. → The scientist was introduced by the artist.
Syntax	Negation	Negate the input sentence.	Time is finite → Time is not finite.
Lexical Semantics	Antonyms	Write a word that means the opposite of the input word.	won → lost
	Synonyms	Write a word with a similar meaning to the input word.	alleged → supposed
	Membership	Write all the animals that appear in the given list.	cat, helicopter, cook, whale, frog, lion → frog, cat, lion, whale
Phonetics	Rhymes	Write a word that rhymes with the input word.	sing → ring
Knowledge	Larger Animal	Write the larger of the two given animals.	koala, snail → koala
Semantics	Cause Selection	Find which of the two given cause and effect sentences is the cause.	Sentence 1: The soda went flat. Sentence 2: The bottle was left open. → The bottle was left open.
	Common Concept	Find a common characteristic for the given objects.	guitars, pendulums, neutrinos → involve oscillations.
Style	Formality	Rephrase the sentence in formal language.	Please call once you get there → Please call upon your arrival.
Numerical	Sum	Sum the two given numbers.	22 10 → 32
	Difference	Subtract the second number from the first.	32 22 → 10
	Number to Word	Write the number in English words.	26 → twenty-six
Multi-lingual	Translation	Translate the word into German / Spanish / French.	game → juego
GLUE	Sentiment Analysis	Determine whether a movie review is positive or negative.	The film is small in scope, yet perfectly formed. → positive
	Sentence Similarity	Rate the semantic similarity of two input sentences on a scale of 0 - definitely not to 5 - perfectly.	Sentence 1: A man is smoking. Sentence 2: A man is skating. → 0 - definitely not
	Word in Context	Determine whether an input word has the same meaning in the two input sentences.	Sentence 1: Approach a task. Sentence 2: To approach the city. Word: approach → not the same

Table 4: The tasks in our instruction-induction experiments. For each task, we show a corresponding instruction and demonstration, with → separating the input from the output.

Reference for Languages (CEFR).[12] Using these words, we create random lists containing between 5 to 7 words, where 3 or 4 are animals and the rest belong to one of the other 5 categories. The induce split is constructed by sampling 3,000 such combinations, using 80% of each category's words. The execute split is constructed by sampling 100 such combinations, using the remaining 20% of each category's words. The task's in-context and execution accuracy are evaluated using an exact *set* match, by treating the output (and y_k) as a set of strings.

Rhymes y_k is a rhyme of the input word x_k. The data was constructed by taking words categorized at the A1, A2, or B1 levels according to CEFR. We then use CMU's pronouncing dictionary[13] to find rhyming groups for these words. The execute split is constructed by sampling 30 rhyming groups, each containing two or more words, and sampling 100 unique words. The induce split is constructed using the rest of the rhyming groups. We evaluate this task by checking whether the predicted word is contained in the rhyming group of x_k.

Larger Animal x_k is two animals, and y_k is the (physically) larger one. We use the object comparison data from oLMpics (Talmor et al., 2020), taking the train split, which only contains animals. We construct the induce set using a sample of 80% of the animals and the execute set by sampling 100 pairs out of the remaining 20% animals.

Cause Selection x_k contains two sentences describing related events, where one event caused the other; y_k contains the cause sentence. As data source, we use the 50 examples from the BIG-bench (Srivastava et al., 2022) *Cause and Effect* task, randomly splitting them to equally-sized induce and execute sets. In each of the induce demonstrations, we randomly sample the position of the cause sentence (either the first or the second sentence in x_k). For examples in the execute set, we take both options for each cause and effect pair, doubling the data.

Common Concept x_k contains a few entities that share a non-trivial common underlying concept, while y_k describes that common concept. We use the 32 examples from *Novel Concepts* in BIG-bench (Srivastava et al., 2022), using half for induce and half for execute. As the BIG-bench answers usually contain clear "task markers" (e.g., answers that start with "They all have...", indicating that the task was to find a common concept), we remove them from our demonstrations. The task's in-context and execution accuracy are evaluated using unigram overlap (F1).

Formality x_k is a sentence in informal English, and y_k is its paraphrase in more formal language. We write 30 sentence pairs ourselves, following existing guidelines for converting informal sentences into formal ones.[14] The task's in-context and execution accuracy are evaluated using unigram overlap (F1).

Sum x_k contains two numbers separated by a space, and y_k is their sum. For each number in the range $[0, 99]$, we enumerate over all pairs.

Difference x_k contains two numbers separated by a space, and y_k is the difference between them. We use all number pairs such that both input numbers are in the range $[0, 198]$, and always subtract the smaller number from the bigger number.

Number to Word x_k is a number written in digits (e.g., 28), and y_k is the same number written in words (e.g, twenty-eight). We use all numbers in range [0,9999].

Translation x_k is an English word and y_k is its translation to some target language – either German, Spanish, or French. We use CEN as input words, and obtain their translations via Wiktionary.[15] For evaluation, we check whether the predicted answer is contained in the set of the possible gold answers.

Sentiment Analysis x_k is a movie review and y_k is a binary label, either "positive" or "negative", marking the review's sentiment. We use the Stanford Sentiment Treebank dataset (Socher et al., 2013) from GLUE (Wang et al., 2018), taking the train split as our induce set and the dev split as the execute set. We consider only full sentences, discarding sentence constituents and sentences containing more than 10 words. This leaves us with

[12] https://languageresearch.cambridge.org/american-english

[13] https://github.com/cmusphinx/cmudict

[14] https://www.niu.edu/writingtutorial/style/formal-and-informal-style.shtml, https://www.uts.edu.au/current-students/support/helps/self-help-resources/grammar/formal-and-informal-language

[15] https://github.com/open-dsl-dict/wiktionary-dict

an induce set of 1,167 examples. To create label-balanced instruction induction examples, we sample each sequence of 5 demonstrations such that there are at least 2 demonstrations for each label.

Sentence Similarity x_k contains two sentences, and y_k reflects the semantic similarity of the two input sentences. The similarity is measured on a scale of 0 to 5, and the labels contain an additional short textual description of the numerical label, e.g., "5 - perfectly". We use the Semantic Textual Similarity Benchmark dataset (Cer et al., 2017) from GLUE, rounding the similarity scores and taking the train split as the induce set and the dev split as the execute set. We discard examples in which at least one of the sentences contains more than 10 words, which leaves us with an induce set of 3,716 examples. In each instruction induction example, we sample at least one pair with a score of 0 and one with a score of 5, so that models will be exposed to the minimal and maximal scores when generating an instruction. We evaluate whether the predicted answer matches one of three valid outputs for each label: the numerical label ("5"), the verbal label ("perfectly"), or the combined label ("5 - perfectly").

Word in Context x_k contains a target word and two contexts (sentences) for that word, and y_k is a binary label reflecting whether the word has the same meaning in both contexts. We use the Word in Context dataset (Pilehvar and Camacho-Collados, 2019) from SuperGLUE (Wang et al., 2019), taking the train split as the induce set and the dev split as the execute set. We discard examples in which at least one of the sentences contains more than 10 words, which leaves us with an induce set of 4,084 examples. To create label-balanced instruction induction examples, we sample each sequence of 5 demonstrations such that there are at least 2 demonstrations for each label. We evaluate whether the predicted label matches one of several possible outputs: "same", "yes", or "true" for an identical meaning, and "not the same", "no", or "false" for a different meaning.

A.3 Common English Nouns

We create a dataset of common English nouns (CEN) by filtering high-frequency nouns from the Wikitext-103 corpus (Merity et al., 2017). We first create a vocabulary of the 10,000 most frequent words in the corpus, from which we will later select the nouns. We then process the corpus with SpaCy's part-of-speech tagger and lemmatizer,[16] and retain only nouns that appear in their singular form by verifying that their part-of-speech tag is "NN" and testing whether the word's lemma is identical to the word itself. We additionally filter nouns that have less than 3 letters. Overall, this leaves us with a set of 3,406 nouns.

Task	In-Context Learning GPT-3	In-Context Learning InstructGPT	Human Study
First Letter	**97**	**98**	**100**
Second Letter	25	34	**100**
List Letters	**98**	**100**	**100**
Starting With	33	46	**80**
Pluralization	**95**	**99**	**100**
Passivization	**100**	**100**	**80**
Negation	**94**	**93**	**100**
Antonyms	**84**	**83**	**100**
Synonyms	9	12	**80**
Membership	13	36	**100**
Rhymes	46	39	**100**
Larger Animal	58	**82**	40
Cause Selection	47	**82**	**100**
Common Concept	23	15	**100**
Formality	54	56	**80**
Sum	**87**	**100**	**100**
Diff	69	**95**	**100**
Number To Word	**85**	**100**	**100**
Translation en-de	**80**	**85**	**100**
Translation en-es	**91**	**88**	**100**
Translation en-fr	**80**	**84**	**80**
Sentiment	**95**	**99**	**100**
Sentence Similarity	3	15	**80**
Word in Context	56	61	**80**

Table 5: Data verification results. The in-context learning scores show how well models can infer our tasks, and the human study scores show how often humans write the correct instruction given the instruction induction prompt. All scores above or equal to 80% are in bold.

B Data Verification

Table 5 shows the results for the data verification experiments (§3.3). As evident by these results, most of our tasks can be inferred in-context by models. Moreover, all tasks but one can be accurately described by at least 4 out 5 human annotators.

Meta-Prompt	First Letter	Passivization	Antonyms	Translation en-de	Sentence Similarity
Challenge Puzzle (Original)	5/5	0/5	1/5	5/5	4/5
Challenge Puzzle + Name	5/5	0/5	2/5	5/5	4/5
Instruction After Demonstrations	5/5	0/5	3/5	5/5	5/5
Instruction Before Demonstrations	5/5	0/5	0/5	2/5	3/5

Table 6: The number of correct instructions generated by text-davinci-002, out of the five examples tested for each task, as inspected for each meta-prompt.

C Meta-Prompt Analysis

As language models are known to be sensitive to the meta-prompt wrapping the demonstrations, we test the instruction induction abilities of the best-performing model, text-davinci-002, when varying the meta-prompt. The instruction induction meta-prompt presented in Figure 1 was selected by showing humans several pre-designed prompts and inspecting which was the clearest for the participants. We test the sensitivity to the meta-prompt by taking three additional meta-prompts (Table 7), sampling five examples from five tasks and manually verifying the correctness of the generated instructions.

Table 6 shows that while the model performance is affected by the content of the meta-prompt, the overall trend is similar when using other meta-prompts, and high performance can be obtained with other prompts as well. In fact, for two of the three additional tested prompts, the generated instructions seem to be even better than those generated using the original prompt, though the differences are too small to determine this conclusively.

Challenge Puzzle (Original)

I gave a friend an instruction and five inputs. The friend read the instruction and wrote an output for every one of the inputs. Here are the input-output pairs:

Input:
Output:
...

The instruction was

Challenge Puzzle + Name

I gave Bob an instruction and five inputs. Bob read the instruction and wrote an output for every one of the inputs. Here are the input-output pairs:

Input:
Output:
...

The instruction was

Instruction After Demonstrations

Below are five input-output pairs that correspond to some underlying task:

Input:
Output:
...

Please write the instruction that best describes the underlying task:

Instruction Before Demonstrations

You are given five examples of input-output pairs. Please write an instruction that describes creating an output from each input.

Input:
Output:
...

Table 7: The meta-prompts used in our analysis.

[16] https://spacy.io/

ACL 2023 Responsible NLP Checklist

A For every submission:

☑ A1. Did you describe the limitations of your work?
3, 9

☒ A2. Did you discuss any potential risks of your work?
One benefit of the proposed approach is better interpretability and explainability, and we therefore view it as a method for reducing risks.

☑ A3. Do the abstract and introduction summarize the paper's main claims?
1

☒ A4. Have you used AI writing assistants when working on this paper?
Left blank.

B ☑ Did you use or create scientific artifacts?

3

☑ B1. Did you cite the creators of artifacts you used?
Appendix A

☒ B2. Did you discuss the license or terms for use and / or distribution of any artifacts?
We verified that all the data and code used is publicly open - we verified license details for each, and we provided citation and links to all relevant resources, where license details can also be found.

☑ B3. Did you discuss if your use of existing artifact(s) was consistent with their intended use, provided that it was specified? For the artifacts you create, do you specify intended use and whether that is compatible with the original access conditions (in particular, derivatives of data accessed for research purposes should not be used outside of research contexts)?
Appendix A

☒ B4. Did you discuss the steps taken to check whether the data that was collected / used contains any information that names or uniquely identifies individual people or offensive content, and the steps taken to protect / anonymize it?
We didn't discuss that, but other than the fact that we only used published datasets that are already used by the research community - we also sampled examples and manually verified their content.

☑ B5. Did you provide documentation of the artifacts, e.g., coverage of domains, languages, and linguistic phenomena, demographic groups represented, etc.?
Appendix A

☑ B6. Did you report relevant statistics like the number of examples, details of train / test / dev splits, etc. for the data that you used / created? Even for commonly-used benchmark datasets, include the number of examples in train / validation / test splits, as these provide necessary context for a reader to understand experimental results. For example, small differences in accuracy on large test sets may be significant, while on small test sets they may not be.
Appendix A

The Responsible NLP Checklist used at ACL 2023 is adopted from NAACL 2022, with the addition of a question on AI writing assistance.

C ☑ Did you run computational experiments?

5

☒ C1. Did you report the number of parameters in the models used, the total computational budget (e.g., GPU hours), and computing infrastructure used?
We used OpenAI models, for which the number of parameters is not always known. For models with known number of parametrs, we did report that number.

☑ C2. Did you discuss the experimental setup, including hyperparameter search and best-found hyperparameter values?
5

☒ C3. Did you report descriptive statistics about your results (e.g., error bars around results, summary statistics from sets of experiments), and is it transparent whether you are reporting the max, mean, etc. or just a single run?
We did not include error bars. The usage of mean values and the number of examples used to calculate the mean are clear and transparent.

☑ C4. If you used existing packages (e.g., for preprocessing, for normalization, or for evaluation), did you report the implementation, model, and parameter settings used (e.g., NLTK, Spacy, ROUGE, etc.)?
4, Appendix A

D ☑ Did you use human annotators (e.g., crowdworkers) or research with human participants?

3

☑ D1. Did you report the full text of instructions given to participants, including e.g., screenshots, disclaimers of any risks to participants or annotators, etc.?
3

☑ D2. Did you report information about how you recruited (e.g., crowdsourcing platform, students) and paid participants, and discuss if such payment is adequate given the participants' demographic (e.g., country of residence)?
3

☑ D3. Did you discuss whether and how consent was obtained from people whose data you're using/curating? For example, if you collected data via crowdsourcing, did your instructions to crowdworkers explain how the data would be used?
3

☒ D4. Was the data collection protocol approved (or determined exempt) by an ethics review board?
The data annotation did not have any associated risks and did not require a special approval.

☑ D5. Did you report the basic demographic and geographic characteristics of the annotator population that is the source of the data?
3